20.11.25 Naji

Task 3: Development Documentation

Phase 1:

```python
"""PHASE 1: IMPORTING LIBRARIES, FILES, INITAL SETUP AND WINDOW"""
import pygame
import random

scream = 'scream.mp3'
nom = 'nom.mp3'

pygame.init()  #Initialises library
pygame.mixer.init() #initialises the sound module in pygame

window = pygame.display.set_mode((800,800))
pygame.display.set_caption("Snake Game")
```

Window: I chose 800 x 800 (640,000 total) as window. I can have a 20 x 18 grid (360 cells). 640,000 / 400 (rounded up) means each cell is made of 1,600 pixels (40 x 40 cell). Saving 2 rows gives me space for score/high score to be displayed.

Phase 2:

```python
"""PHASE 2: DEFINING ALL FUNCTIONS"""
def checkered():    #Creates checkered background
    tile_size = 40
    for row in range(18):
        for col in range(20):
            if (row + col) % 2 == 0:
                color = (160, 160, 160)  # Light grey
            else:
                color = (96, 96, 96)     # Dark grey
            pygame.draw.rect(window, color, (col * tile_size, row * tile_size + 80, tile_size, tile_size))

def load_high_score():
    try:
        with open("highscore.txt", "r") as file:
            return int(file.read())
    except:
        return 0 # if file doesn't exist or empty it returns 0. Error handling
```

Checkered: Tile size sets each grid cell to be 40 x 40 pixels. For checkered design I used nested loops for rows and columns. To create a checkered pattern, I used a modulator (%) to get even numbered squares light grey and odd numbered squares dark grey (alternating). I chose row 18 and column 20 to allow for scores/high scores to be visible above. (Y – offset = tile_size[40] + 80).

High score: Added error handling if highscore.txt file wasn't found. And an example of file I/O using empty text file.

```python
def game_over_msg():
    window.fill((0, 0, 135), (0, 80, 800, 720))
    font_large = pygame.font.Font(None, 74)
    text = font_large.render(f"Game Over! Score:{score}", True, (255, 255, 255))
    high_score_text = font_large.render(f"High Score: {high_score}", True, (255, 255, 255))
    text_rect = text.get_rect(center = (400, 300))
    hstext_rect = high_score_text.get_rect(center = (300, 200))

    window.blit(text, text_rect)
    window.blit(high_score_text, hstext_rect)

    font_small = pygame.font.Font(None, 36)
    restart_text = font_small.render("Press ENTER to restart", True, (255, 255, 255))
    restart_rect = restart_text.get_rect(center = (400,400))
    window.blit(restart_text, restart_rect)
    pygame.display.update()
```

Game over message: Dark blue (0, 0, 135) fills entire screen except top black box for score (0, 80, 800, 720). All text is in white (255, 255, 255). Text for score, high score and restart option.

Took some trial and error to display the messages neatly without overlap.

20.11.25 Naji

```python
"""PHASE 3: GAME PARAMETERS"""
score = 0
high_score = load_high_score()

snake = [(5, 9), (6, 9), (7, 9)]  #initial snake position
obstacles = [] #coordinates of blocks after scoring stored as list

direction = "RIGHT"      #starting direction
dx, dy = 1, 0

clock = pygame.time.Clock()    #game clock

game_over = False

#Setting up burger/food item
burger_img = pygame.image.load("burger.png").convert_alpha()
burger_img = pygame.transform.scale(burger_img, (40, 40))
burger_x = random.randint(0, 20)
burger_y = random.randint(0, 18)
```

Snake variable stored as starting x,y positions in a list. Set up score and high score, game starts with no obstacles so the list is initially empty. Snake moves initially moving right from the middle of the grid. Loaded the burger image from .png file in this phase and downsized it to make it fit in the squares.

The main game loop is the main part of the game that handles:

- User input/Controls causing change of x,y (movement logic)
- Drawing checkered background
- Score/High score and instructions to play displayed.
- Drawing of snake
- Collision checks with the burger, wall or obstacle.
- Spawning of new burger, increase in score, high score logic etc

```python
"""PHASE 4: MAIN GAME LOOP"""
running = True        #Main game loop
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RIGHT or event.key == pygame.K_d:
                dx, dy = 1, 0
                direction = "RIGHT"
            elif event.key == pygame.K_LEFT or event.key == pygame.K_a:
                dx, dy = -1, 0
                direction = "LEFT"
            elif event.key == pygame.K_UP or event.key == pygame.K_w:
                dx, dy = 0, -1
                direction = "UP"
            elif event.key == pygame.K_DOWN or event.key == pygame.K_s:
                dx, dy = 0, 1
                direction = "DOWN"

    checkered()
```

Event handling is receiving user input (To close the game or pressing arrows, WASD for controlling snake).

```python
    font = pygame.font.Font(None, 36)
    font_s = pygame.font.Font(None, 24)
    score_bg = pygame.Rect(0, 0, 800, 80)
    pygame.draw.rect(window, (0, 0, 0), score_bg)
    score_text = font.render(f"Score: {score}", True, (255,255,255))
    high_score_text = font.render(f"High Score: {high_score}", True, (255, 255, 255))
    window.blit(score_text, (20,20))
    window.blit(high_score_text, (240, 20))

    controls = font_s.render("Use Arrows or WASD for movement", True, (255, 255, 255))
    window.blit(controls, (500, 20))

    #draws burger on grid
    window.blit(burger_img, (burger_x * 40, burger_y * 40 + 80))

    for obs_x, obs_y in obstacles:
        pygame.draw.rect(window, (102, 0, 204), (obs_x * 40, obs_y * 40 + 80, 40, 40))

    #Logically --> Movement (1st)
    head_x, head_y = snake[0] #initial head position
    new_head = (head_x + dx, head_y + dy) #new head position
    snake.insert(0, new_head)
```

102, 0, 204 (purple) obstacles drawn based on x and y coordinates.

The new head is updated on the first item of the snake list. snake.insert(0, new_head).

```python
    #Allows length of snake to increase by 1 if burger eaten.
    if not (new_head[0] == burger_x and new_head[1] == burger_y):
        snake.pop()

    #Collision checks for burger (2nd)
    if new_head[0] == burger_x and new_head[1] == burger_y:
        score += 1
        pygame.mixer.music.load(nom)
        pygame.mixer.music.play()
        valid_burger = False
        while not valid_burger:
            burger_x = random.randint(0, 19)
            burger_y = random.randint(0, 17)
            if (burger_x, burger_y) not in obstacles:
                valid_burger = True

        #spawning of obstacle
        obstacle_spwn = (random.randint(0, 19), random.randint(0, 17))
        obstacles.append(obstacle_spwn)
```

Checks if snake head is on the burger returns True if snake ate burger False if not causes snake.pop() removes last element (tail) which keeps same length when moving.

Wrote this line to prevent the burger from spawning behind the obstacle making it impossible for player to progress

```python
    #Drawing the snake (3rd)
    for i, (x, y) in enumerate(snake):
        pixel_x = x * 40
        pixel_y = y * 40 + 80

        if i == 0: #head
            color = (0, 204, 102)
        elif i == len(snake) - 1: #tail
            color = (255, 255, 51)
        else: #body
            color = (51, 255, 51)

        pygame.draw.rect(window, color, (pixel_x, pixel_y, 40, 40))

    #Wall collision check (4th)
    if (new_head[0] < 0 or new_head[0] >= 20 or new_head[1] < 0 or new_head[1] >= 18):
        pygame.mixer.music.load(scream)
        pygame.mixer.music.play()
        game_over = True

    #Obstacle collision check (5th)
    if new_head in obstacles:
        game_over = True
```

In main game loop for statement used to index starting position (head[0]), tail (length of snake -1 [last index]) using enumerate(). For statement converts positions into x/y squares to draw.

Using len(snake) – 1 is flexible when snake grows in length.

Assigned head, tail and body different colours Yellow tail, Light green body

```python
    #Gamover due to any collision (6th)
    if game_over:
        while game_over:
            game_over_msg()

            #Game over screen loop
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    game_over = False
                    running = False
                elif event.type == pygame.KEYDOWN:
                    if event.key == pygame.K_RETURN:  #restarts game
                        if score > high_score:
                            with open("highscore.txt", "w") as file:
                                file.write(str(score))
                                high_score = score
                        snake = [(5,9), (6,9), (7,9)]
                        dx, dy = 1, 0
                        score = 0
                        obstacles = []
                        game_over = False

    pygame.display.update()  #draws entire display

    clock.tick(10) #10 fps for snake movement

pygame.quit()
```

Why did I not choose to go with Object Oriented Programming (OOP)?

When I first started the project I was simply working step by step, including each component after the other when I realised I could have used OOP instead.

I tried making a separate version that used OOP but found it to be too complex due to repeating certain words and so I decided to stick with what I already had started.

For me this way of writing the game made more sense to me because it's split into individual tasks that need to be implemented in the game.

Reference list

- Ashot (2011). *How can I play an mp3 with pygame?* [online] Stack Overflow. Available at: https://stackoverflow.com/questions/7746263/how-can-i-play-an-mp3-with-pygame [Accessed 1 Dec. 2025].
- evildie.x (2021). *Wilhelm Scream - Sound Effect*. [online] www.youtube.com. Available at: https://www.youtube.com/watch?v=PP7WJL2JtLs.
- magdum (2020). *32x32 food set*. [online] OpenGameArt.org. Available at: https://opengameart.org/content/32x32-food-set [Accessed 24 Nov. 2025].
- Mertz, J. (2024). *7. Input and Output*. [online] Python documentation. Available at: https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files [Accessed 1 Dec. 2025].
- RapidTables (2019). *RGB Color Codes Chart*. [online] Rapidtables.com. Available at: https://www.rapidtables.com/web/color/RGB_Color.html [Accessed 20 Nov. 2925].
- Stack Overflow. (n.d.). *Drawing a checkerboard in Python*. [online] Available at: https://stackoverflow.com/questions/32704485/drawing-a-checkerboard-in-python [Accessed 20 Nov. 2025].
- Team Fortress 2 - All Sounds (2017). *'"Nom nom nom, om nom!"' (Heavy Voice Lines)*. [online] YouTube. Available at: https://www.youtube.com/watch?v=kO1Yq7n9zLU [Accessed 24 Nov. 2025].
- www.pygame.org. (n.d.). *pygame.key — pygame v2.6.0 documentation*. [online] Available at: https://www.pygame.org/docs/ref/key.html#pygame.key.get_pressed [Accessed 22 Nov. 2025].
- YT1s. (2025). *YouTube to MP3 Converter - YT1s*. [online] Available at: https://yt1s.com.co/en157/youtube-to-mp3/ [Accessed 24 Nov. 2025].